

Tema 07 · JSTL y Cookies

Clase 021 (09-09) · Teoría: «JSP» (Clase020_jsp.pdf): cookies y JSP Tags

Programación Avanzada · Prof. Macrino Matías · Universidad del Salvador – Sede Pilar
· 2.º cuatrimestre

1. Cookies

Una **cookie** es un pequeño dato de texto (clave = valor) que el **servidor envía al navegador** y que el navegador **guarda y devuelve** en cada petición posterior. A diferencia de la Session (datos en el servidor), la cookie guarda el dato **en el cliente**. Usos: recordar la última búsqueda, el idioma, presentar el sitio como el usuario lo dejó, y **recordar al visitante sin que vuelva a loguearse** (el requisito de «sesión recordada» del parcial).

Acción	Código Java (Servlet o JSP)
Crear una cookie	<code>Cookie c = new Cookie("ultimoDestino", valor);</code>
Definir su vida (segundos)	<code>c.setMaxAge(60*60*24*7);</code> → 7 días. Sin <code>setMaxAge</code> dura solo hasta cerrar el navegador. 0 = borrarla
Enviarla al navegador	<code>response.addCookie(c);</code>
Leer las cookies que llegan	<code>Cookie[] cookies = request.getCookies();</code> (puede ser null si no hay ninguna) y recorrerlas con <code>for</code> , comparando <code>getName()</code> y leyendo <code>getValue()</code>
Borrar una cookie	Crear una con el mismo nombre, setMaxAge(0) y enviarla con <code>addCookie</code>

Seguridad: los valores de las cookies no admiten ciertos caracteres (espacios, tildes, punto y coma), por eso se codifican con `URLEncoder.encode(valor, "UTF-8")` al guardar (y se decodifican con `URLDecoder.decode` al leer). Nunca se guarda una contraseña en una cookie; para recordar al usuario se guarda un **token aleatorio**. Atributos importantes: **HttpOnly** (JavaScript no puede leerla), **Secure** (solo por HTTPS) y **SameSite**.

	Session	Cookie
Dónde se guardan los datos	En el servidor	En el navegador (cliente)
Qué viaja	Solo el id (JSESSIONID, que es una cookie)	El dato completo, en cada petición
Duración	Hasta <code>invalidate()</code> o <code>timeout</code>	Hasta la fecha de vencimiento (<code>maxAge</code>)
Seguridad	Mayor (el dato no sale del servidor)	Menor (el usuario puede verla y modificarla)

2. JSTL: la librería de etiquetas estándar

JSTL (JSP Standard Tag Library) es una librería de etiquetas con forma de XML que **reemplaza a los scriptlets**. Ventajas: el JSP queda más limpio y legible, se separa la presentación de la lógica (código Java fuera de la vista) y se mantiene más fácil. Se compone de: **Core** (lógica: `if`, `forEach`, `choose`, `set`, `out`, `redirect`), **Formatting** (formato de números, fechas y mensajes), **SQL**, **XML** y **Functions** (funciones de texto como `fn:escapeXml`).

Cómo se usa: (1) agregar las dependencias al pom (jstl y standard); (2) declarar la librería en cada JSP con la directiva **taglib** y un prefijo:

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt" %>
```

Etiqueta	Equivale a	Ejemplo
<c:if>	if (sin else)	<code><c:if test="\${ not empty lista}"> ... </c:if></code>
<c:forEach>	for / for-each	<code><c:forEach var="p" items="\${ productos}"> ... </c:forEach></code>
<c:choose> / <c:when> / <c:otherwise>	if / else if / else (switch)	<code><c:when test="\${ p.enStock}"></code>
<c:set> / <c:out>	asignar variable / imprimir (escapando HTML)	<code><c:out value="\${ nombre}"/></code>
<fmt:formatNumber>	formatear un número	<code><fmt:formatNumber value="\${ p.precio}" type="currency" currencySymbol="\$"/></code>

Expression Language (EL): \${ }

JSTL se usa junto con el **EL**, una sintaxis corta para leer datos: **\${producto.nombre}** equivale a `producto.getNombre()` (llama al getter; por eso las clases deben seguir la convención JavaBean: `getXxx/isXxx`). Para los booleanos el getter es **isEnStock()** y en EL se escribe **\${producto.enStock}**. EL busca la variable en los alcances `page` → `request` → `session` → `application`. Operadores útiles: **empty** (null o vacío), **not**, **and**, **or**, **==**.

Muy importante para el parcial: en el código de clase (Tomcat 9, `javax.servlet`) el URI de JSTL es `http://java.sun.com/jsp/jstl/core`. En Tomcat 10 en adelante (Jakarta EE) cambia a **jakarta.tags.core**, y las dependencias también. La solución del parcial incluida en este repositorio usa Jakarta; si el profesor usa Tomcat 9 hay que adaptar esos nombres (ver advertencia en el README).

3. Scriptlets vs. JSTL (comparación)

La clase muestra la **misma tabla de productos hecha de dos maneras**: con scriptlets (for de Java y `out.println` para escribir cada fila: código difícil de leer, mezcla HTML con Java) y con JSTL (`c:forEach` y `c:choose` dentro del HTML: legible y fácil de mantener). Esta comparación es exactamente la justificación del punto «Uso de JSP y tags JSP (JSTL)» del parcial.

Código de las clases, explicado línea por línea

Cada recuadro gris reproduce líneas reales del archivo (con su número de línea) y el recuadro verde de abajo explica qué hacen y por qué están escritas así.

Clase 021 - 09-09: cookies (último destino buscado) y tabla de productos con scriptlets y con JSTL

Tres ejercicios: (1) **destinos.jsp**, **procesamiento.jsp** y **eliminar.jsp** guardan, leen y borran en una **cookie** el último destino elegido; (2) **tabla_scriptlets.jsp** arma una tabla de productos con Java; (3) **tabla_jstl.jsp** hace la misma tabla con **JSTL**. La clase **Producto** es el modelo.

021 - Clase 09-09 / pom.xml

Qué es y para qué sirve: Archivo de configuración de Maven (POM) de una aplicación web: coordenadas, dependencias y nombre del WAR.

Cómo defenderlo / qué decir: Es el POM que genera el arquetipo de Maven para aplicaciones web. Lo importante: **packaging war**, las APIs de Servlet/JSP como **provided** (las aporta el contenedor) y, si la página usa JSTL, esas librerías sí van en el WAR. Esta versión incluye JSTL.

```
1 | <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
2 |   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
```

Líneas 1 a 2: Cabecera del POM: elemento raíz **project** con sus espacios de nombres XML (definen el formato válido del archivo).

```
3 |   <modelVersion>4.0.0</modelVersion>
```

Línea 3: Versión del modelo de POM (siempre 4.0.0).

```
4 |   <groupId>edu.usal</groupId>
5 |   <artifactId>clase-09-09</artifactId>
6 |   <packaging>war</packaging>
7 |   <version>1.0-SNAPSHOT</version>
8 |   <name>clase-09-09 Maven Webapp</name>
9 |   <url>http://maven.apache.org</url>
```

Líneas 4 a 9: Coordenadas del proyecto: **groupId** edu.usal (la organización), **artifactId** clase-09-09 (el nombre), **packaging war** (se empaqueta como aplicación web, un archivo .war que se despliega en Tomcat), versión 1.0-SNAPSHOT, nombre descriptivo y URL (datos del arquetipo maven-archetype-webapp).

```
10 |   <dependencies>
```

Línea 10: Abre la lista de dependencias (librerías que Maven descarga solo).

```
11 |     <dependency>
12 |       <groupId>junit</groupId>
13 |       <artifactId>junit</artifactId>
14 |       <version>3.8.1</version>
15 |       <scope>test</scope>
16 |     </dependency>
17 |
```

Líneas 11 a 17: **JUnit 3.8.1** con **scope test**: solo se usa para pruebas unitarias, no viaja dentro del WAR.

```
18 |     <dependency>
19 |       <groupId>javax.servlet</groupId>
20 |       <artifactId>javax.servlet-api</artifactId>
21 |       <version>4.0.1</version>
22 |       <scope>provided</scope>
23 |     </dependency>
24 |
```

Líneas 18 a 24: **javax.servlet-api 4.0.1** con **scope provided**: se necesita para compilar los servlets, pero en ejecución la provee el servidor (Tomcat 9), por eso no se incluye en el WAR.

```

25 | <dependency>
26 |   <groupId>javax.servlet.jsp</groupId>
27 |   <artifactId>javax.servlet.jsp-api</artifactId>
28 |   <version>2.3.3</version>
29 |   <scope>provided</scope>
30 | </dependency>
31 |

```

Líneas 25 a 31: javax.servlet.jsp-api 2.3.3, también **provided**: la API de JSP que aporta Tomcat.

```

32 | <dependency>
33 |   <groupId>javax.servlet</groupId>
34 |   <artifactId>jstl</artifactId>
35 |   <version>1.2</version>
36 | </dependency>
37 |

```

Líneas 32 a 37: jstl 1.2 (scope compile por defecto): la implementación de JSTL; **Tomcat no la trae**, por eso debe ir dentro del WAR (carpeta WEB-INF/lib).

```

38 | <dependency>
39 |   <groupId>javax.servlet.jsp.jstl</groupId>
40 |   <artifactId>jstl-api</artifactId>
41 |   <version>1.2</version>
42 | </dependency>
43 |

```

Líneas 38 a 43: jstl-api 1.2: la API (interfaces) de JSTL.

```

44 | <dependency>
45 |   <groupId>>taglibs</groupId>
46 |   <artifactId>standard</artifactId>
47 |   <version>1.1.2</version>
48 | </dependency>

```

Líneas 44 a 48: taglibs standard 1.1.2: las librerías de etiquetas estándar (los .tld con core, fmt, etc.).

```

49 | </dependencies>

```

Línea 49: Cierre de la lista de dependencias.

```

50 | <build>
51 |   <finalName>clase-09-09</finalName>
52 | </build>

```

Líneas 50 a 52: finalName define el nombre del WAR generado (clase-09-09.war) y por lo tanto la **ruta de contexto** de la aplicación: http://localhost:8080/clase-09-09/.

```

53 | </project>

```

Línea 53: Cierre del POM.

021 - Clase 09-09 / src / main / webapp / WEB-INF / web.xml

Qué es y para qué sirve: web.xml: descriptor de despliegue de la aplicación web. Vive en WEB-INF y es el archivo de configuración que lee el contenedor (Tomcat) al iniciar.

Cómo defenderlo / qué decir: Aquí solo contiene el nombre de la aplicación porque todavía no hay servlets declarados por XML. Es el lugar donde se declararían servlets, filtros, páginas de error y configuración de sesión (ver Tema 08).

```

1 | <!DOCTYPE web-app PUBLIC
2 |   "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
3 |   "http://java.sun.com/dtd/web-app_2_3.dtd" >
4 |

```

Líneas 1 a 4: Declaración del tipo de documento (**DOCTYPE**): indica que es un descriptor de despliegue de aplicación web.

```

5 | <web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
6 |     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
7 |     xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
8 |                         http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
9 |     version="4.0">

```

Líneas 5 a 9: Elemento raíz **web-app** con los espacios de nombres de Java EE y **version="4.0"** (Servlet 4.0): habilita funciones modernas como las **anotaciones** (@WebServlet) y el EL de JSP 2.3.

```

10 | <display-name>Archetype Created Web Application</display-name>

```

Línea 10: Nombre descriptivo de la aplicación (lo muestra el administrador de Tomcat).

```

11 | </web-app>

```

Línea 11: Cierre del descriptor.

021 - Clase 09-09 / src / main / java / edu / usal / model / Producto.java

Qué es y para qué sirve: Clase de **modelo** (JavaBean) que representa un producto: nombre, precio y si hay stock.

Cómo defenderlo / qué decir: Sigue la **convención JavaBean**: atributos privados + getters/setters públicos. Esto es lo que permite a EL leer \${producto.nombre}: EL llama a getNombre(). Para el booleano el getter se llama **isEnStock()** (convención de boolean) y en EL se usa \${producto.enStock}.

```

1 | package edu.usal.model;
2 |

```

Líneas 1 a 2: Paquete **edu.usal.model**: el modelo de datos.

```

3 | public class Producto {
4 |
5 |     private String nombre;
6 |     private double precio;
7 |     private boolean enStock;
8 |

```

Líneas 3 a 8: Tres atributos **privados** (encapsulamiento): nombre (String), precio (double) y enStock (boolean).

```

9 |     public Producto(String nombre, double precio, boolean enStock) {
10 |         this.nombre = nombre;
11 |         this.precio = precio;
12 |         this.enStock = enStock;
13 |     }
14 |

```

Líneas 9 a 14: Constructor con todos los datos: asigna cada parámetro al atributo con **this**.

```

15 |     public String getNombre() {
16 |         return nombre;
17 |     }
18 |
19 |     public void setNombre(String nombre) {
20 |         this.nombre = nombre;
21 |     }
22 |

```

Líneas 15 a 22: Getter y setter del nombre.

```

23 |     public double getPrecio() {
24 |         return precio;
25 |     }
26 |
27 |     public void setPrecio(double precio) {
28 |         this.precio = precio;
29 |     }
30 |

```

Líneas 23 a 30: Getter y setter del precio.

```

31 |     public boolean isEnStock() {
32 |         return enStock;
33 |     }
34 |
35 |     public void setEnStock(boolean enStock) {
36 |         this.enStock = enStock;
37 |     }
38 | }

```

Líneas 31 a 38: Para los **boolean** el getter se llama **isXxx** (JavaBean) y el setter, **setEnStock**.

021 - Clase 09-09 / src / main / webapp / destinos.jsp

Qué es y para qué sirve: Página que **lee la cookie** «ultimoDestino» y la muestra; contiene el formulario para elegir un destino y el botón para eliminar el guardado.

Cómo defenderlo / qué decir: Muestra la lectura de cookies: **getCookies()** puede ser null (primera visita), se recorre el arreglo buscando por nombre y se sale con **break** al encontrarla. Queda comentada la alternativa con **Session**: así se aprecia la diferencia entre ambas. **Observaciones:** el valor se guardó codificado con **URLEncoder**, así que al leerlo conviene decodificarlo con **URLDecoder.decode** (con «cancún» se vería «canc%C3%BA»); el select tiene una errata en el atributo (**id=selectDestino**", falta la comilla de apertura); y la página no incluye el CSS de Bootstrap, por lo que las clases no tienen efecto.

```

1 | <%--
2 |   Created by IntelliJ IDEA.
3 |   User: usal
4 |   Date: 9/9/26
5 |   Time: 7:16 p. m.
6 |   To change this template use File | Settings | File Templates.
7 | --%>

```

Líneas 1 a 7: Comentario JSP generado por el IDE (autor y fecha); no llega al navegador.

```

8 | <%@ page contentType="text/html; charset=UTF-8" language="java" %>

```

Línea 8: Directiva page: tipo de contenido y codificación UTF-8.

```

9 | <html>
10 | <head>
11 |   <meta charset="UTF-8">
12 |   <title>Destinos JSP</title>
13 |   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
14 |     integrity="sha384-sRIL4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuquIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
15 | </head>

```

Líneas 9 a 14: Apertura de html y head (charset y título).

```

15 | <%
16 |   //String ultimoDestino = (String) session.getAttribute("ultimoDestino");
17 |   String ultimoDestino = "";
18 |   Cookie[] cookies = request.getCookies();
19 | %>

```

Líneas 15 a 19: Scriptlet: la primera línea comentada obtenía el destino de la Session. Se declara **ultimoDestino** vacío y se piden las cookies con **request.getCookies()**.

```

20 |     if(cookies != null) {
21 |         for(Cookie cookie : cookies) {
22 |             if("ultimoDestino".equals(cookie.getName())){
23 |                 ultimoDestino = cookie.getValue();
24 |                 break;
25 |             }
26 |         }
27 |     }
28 |
29 | %>

```

Líneas 20 a 29: Si hay cookies (si no, el arreglo es **null**), las recorre con **for-each**; si el nombre es «ultimoDestino» guarda su valor con **getValue()** y corta con **break**. Se escribe "ultimoDestino".equals(...) para evitar **NullPointerException**.

```

30 | <body>
31 |
32 |     <div class="container">
33 |
34 |         <div class="card-header">
35 |             <p class="alert alert-info">
36 |                 Tu ultima busqueda guardada fue :
37 |                 <strong>
38 |                     <%= ultimoDestino %>
39 |                 </strong>
40 |             </p>
41 |         </div>
42 |
43 |         <div class="card-body">

```

Líneas 30 a 43: Cuerpo: un aviso informativo que imprime con una **expresión** el último destino guardado (vacío si no hay).

```

44 |         <form action="procesamiento.jsp" method="post">
45 |             <div class="form-label">
46 |                 <label for="selectDestino">¿Donde te gustaria ir?</label>
47 |             </div>
48 |             <select id="selectDestino" name="selectDestino" class="form-select">
49 |                 <option value="">Seleccione un destino...</option>
50 |                 <option value="bariloche">Bariloche</option>
51 |                 <option value="madrid">Madrid</option>
52 |                 <option value="cancún">Cancun</option>
53 |             </select>
54 |             <button type="submit" class="mb-2 btn btn-primary">Buscar </button>
55 |         </form>
56 |     </div>
57 |
58 | </div>

```

Líneas 44 a 58: Formulario **POST** a procesamiento.jsp con una lista desplegable (name="selectDestino") con tres destinos y el botón «Buscar».

```

59 |         <form method="post" action="eliminar.jsp">
60 |             <button type="submit" class="btn btn-danger">Eliminar destino guardado</button>
61 |         </form>
62 |     </div>
63 | </div>
64 |
65 | </body>
66 | </html>

```

Líneas 59 a 66: Segundo formulario POST, con el botón «Eliminar destino guardado» que llama a eliminar.jsp.

021 - Clase 09-09 / src / main / webapp / procesamiento.jsp

Qué es y para qué sirve: JSP **sin interfaz** que recibe el destino elegido, lo **guarda en una cookie** y vuelve a destinos.jsp.

Cómo defenderlo / qué decir: Es el ejemplo de **crear una cookie**: new Cookie, setMaxAge, response.addCookie. El valor se codifica con **URLEncoder** porque las cookies no admiten ciertos caracteres. Termina con **sendRedirect**: es el patrón Post/Redirect/Get (después de procesar se redirige para que al refrescar no se reenvíe el formulario).

```

1 | <%@ page import="java.util.Locale" %>
2 | <%@ page import="java.net.URLEncoder" %><%--
3 |     Created by IntelliJ IDEA.
4 |     User: usal
5 |     Date: 9/9/26
6 |     Time: 7:30 p. m.
7 |     To change this template use File | Settings | File Templates.
8 | --%>

```

Líneas 1 a 8: Directivas **page import**: Locale (sin usar) y **URLEncoder**.

```

9 | <%@ page contentType="text/html; charset=UTF-8" language="java" %>
10 | <html>
11 | <head>
12 |     <title>Title</title>
13 | </head>

```

Líneas 9 a 13: Directiva **page** (UTF-8). Luego html, head y título.

```

14 | <%
15 |     String selectDestino = request.getParameter("selectDestino");
16 |

```

Líneas 14 a 16: Scriptlet principal. Lee el parámetro **selectDestino** con `getParameter`.

```

17 |         if(selectDestino != null && !selectDestino.trim().isEmpty()){
18 |             String strSanitizado = URLEncoder.encode(selectDestino, "UTF-8");
19 |

```

Líneas 17 a 19: Si el destino no es null ni vacío (**trim()** quita espacios) lo procesa: **URLEncoder.encode(..., "UTF-8")** lo «sanitiza» (codifica tildes, espacios y símbolos).

```

20 |             //Crear la cookie
21 |             Cookie cookie = new Cookie("ultimoDestino", strSanitizado);
22 |
23 |             cookie.setMaxAge(60 * 60 * 24 * 7);
24 |
25 |             //Agregarla al response
26 |             response.addCookie(cookie);
27 |         }
28 |

```

Líneas 20 a 28: Crea la cookie **ultimoDestino** con el valor codificado y define que vive **7 días** (60 s × 60 min × 24 h × 7), y la agrega a la respuesta con **response.addCookie**.

```

29 |         //session.setAttribute("ultimoDestino", selectDestino);
30 |
31 |         response.sendRedirect("destinos.jsp");
32 |     %>

```

Líneas 29 a 32: Línea comentada con la alternativa por Session. Luego **response.sendRedirect("destinos.jsp")**: indica al navegador que pida esa página (respuesta 302).

```

33 | <body>
34 |
35 | </body>
36 | </html>

```

Líneas 33 a 36: Body vacío: esta página nunca se ve, solo procesa y redirige.

021 - Clase 09-09 / src / main / webapp / eliminar.jsp

Qué es y para qué sirve: JSP sin interfaz que **borra la cookie** del último destino y vuelve a `destinos.jsp`.

Cómo defenderlo / qué decir: Una cookie no se «borra» directamente: se envía una cookie del **mismo nombre** con **maxAge 0** y el navegador la descarta. Las líneas comentadas muestran las opciones equivalentes para la Session (`setAttribute null`, `invalidate`, `removeAttribute`). Este es el mismo mecanismo que usa el cierre de sesión del parcial para limpiar las cookies.

```

1 | <!--
2 |     Created by IntelliJ IDEA.
3 |     User: usal
4 |     Date: 9/9/26
5 |     Time: 7:30 p. m.
6 |     To change this template use File | Settings | File Templates.
7 | -->

```

Líneas 1 a 7: Comentario JSP generado por el IDE (autor y fecha); no llega al navegador.

```

8 | <%@ page contentType="text/html; charset=UTF-8" language="java" %>
9 | <html>
10 | <head>
11 |     <title>Title</title>
12 | </head>

```

Líneas 8 a 12: Directiva page (UTF-8), html, head y título.

```

13 | <%
14 |     //session.setAttribute("ultimoDestino", null);
15 |     //session.invalidate();
16 |     //session.removeAttribute("ultimoDestino");
17 |

```

Líneas 13 a 17: Scriptlet. Las tres primeras líneas (comentadas) son las formas de olvidar el dato si estuviera en la **Session**.

```

18 |     Cookie cookie = new Cookie("ultimoDestino", "");
19 |
20 |     cookie.setMaxAge(0);
21 |
22 |     response.addCookie(cookie);
23 |

```

Líneas 18 a 23: Crea una cookie con el **mismo nombre** y valor vacío; **setMaxAge(0)** le indica al navegador que la elimine ya; **addCookie** la envía en la respuesta.

```

24 |     response.sendRedirect("destinos.jsp");
25 | %>

```

Líneas 24 a 25: Redirige a destinos.jsp, que ahora mostrará el destino vacío.

```

26 | <body>
27 |
28 | </body>
29 | </html>

```

Líneas 26 a 29: Body vacío.

021 - Clase 09-09 / src / main / webapp / tabla_scriptlets.jsp

Qué es y para qué sirve: Tabla de productos armada **con scriptlets**: la forma «a la antigua».

Cómo defenderlo / qué decir: Sirve de contraste con la versión JSTL: acá hay que usar **out.println** para escribir cada etiqueta HTML desde Java: el código es largo, difícil de leer y mezcla lógica con presentación. Además la lista se crea dentro del propio JSP (en una aplicación real vendría de un DAO a través del servlet).

```

1 | <%@ page import="java.util.List" %>
2 | <%@ page import="java.util.ArrayList" %>
3 | <%@ page import="edu.usal.model.Producto" %><!--
4 |     Created by IntelliJ IDEA.
5 |     User: usal
6 |     Date: 9/9/26
7 |     Time: 8:19 p. m.
8 |     To change this template use File | Settings | File Templates.
9 | --%>

```

Líneas 1 a 9: Directivas **page import** de List, ArrayList y de la clase **Producto** del modelo.

```

10 | <%@ page contentType="text/html; charset=UTF-8" language="java" %>
11 | <html>
12 | <head>
13 |     <title>Tabla con Scriptlets</title>
14 |     <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
15 |         integrity="sha384-sRIL4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
16 | </head>

```

Líneas 10 a 15: Directiva page (UTF-8). Luego el comentario del IDE, html, head y título.

```

16 | <%

```

Línea 16: Scriptlet: crea una lista de productos y agrega tres productos de ejemplo.

```

17 |     List<Producto> productos = new ArrayList<>();
18 |     productos.add(new Producto("Teclado Mecanico", 45600.50, true));
19 |     productos.add(new Producto("Mouse Inalambrico", 12800.99, true));
20 |     productos.add(new Producto("Monitor 24' Widescreen", 210000.50, true));
21 | %>

```

Líneas 17 a 21: Se declara **List<Producto>** con **ArrayList** y se agregan con **add(new Producto(...))**.

```

22 | <body>
23 |   <div class="container">
24 |     <h3 class="text-danger mb-3">Lista de productos</h3>
25 |
26 |     <table class="table table-border bg-white shadow-sm">
27 |       <thead class="table-dark">
28 |         <tr>
29 |           <th>Producto</th>
30 |           <th>Precio</th>
31 |           <th>Estado</th>
32 |         </tr>
33 |       </thead>

```

Líneas 22 a 33: Cuerpo: contenedor, título y apertura de la tabla con su encabezado (**thead** con las columnas Producto, Precio y Estado).

```

34 |       <tbody>
35 |         <%

```

Líneas 34 a 35: Cuerpo de la tabla.

```

36 |             for(Producto producto : productos) {
37 |                 out.println("<tr>");
38 |                 out.println("<td>");
39 |                 out.println(producto.getNombre());
40 |                 out.println("</td>");
41 |                 out.println("<td>");
42 |                 out.println(producto.getPrecio());
43 |                 out.println("</td>");
44 |                 out.println("<td>");
45 |                 out.println(producto.isEnStock());
46 |                 out.println("</td>");
47 |                 out.println("</tr>");
48 |             }
49 |         }

```

Líneas 36 a 49: **for-each** de Java dentro de un scriptlet: por cada producto escribe con **out.println** cada etiqueta (tr, td, el dato, /td...): nombre, precio y si está en stock (se verá true o false).

```

50 |             </tbody>
51 |         </table>
52 |     </div>
53 |
54 | </body>
55 | </html>

```

Líneas 50 a 55: Cierra la tabla, el contenedor, body y html.

021 - Clase 09-09 / src / main / webapp / tabla_jstl.jsp

Qué es y para qué sirve: La **misma tabla de productos** pero con **JSTL** y **EL**: la forma recomendada.

Cómo defenderlo / qué decir: Es el modelo de vista que pide el parcial. Puntos para defender: **(1)** la directiva **taglib** con prefijo c y fmt; **(2)** `pageContext.setAttribute` hace visible la lista para EL (EL solo ve atributos de page/request/session/application, no variables locales del scriptlet); **(3)** `c:if` con empty para el mensaje de lista vacía; **(4)** `c:forEach` para recorrer; **(5)** `fmt:formatNumber` para mostrar precios como moneda; **(6)** `c:choose` para el estado con colores. El scriptlet que arma la lista está sólo para la demostración: en una aplicación real el **servlet** deja la lista en el request y la JSP no tiene nada de Java (Tema 08).

```

1 | <%@ page import="java.util.List" %>
2 | <%@ page import="java.util.ArrayList" %>
3 | <%@ page import="edu.usal.model.Producto" %>

```

Líneas 1 a 3: Directivas **page import** (List, ArrayList y Producto).

```

4 | <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
5 | <%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt" %><%-
6 | Created by IntelliJ IDEA.
7 | User: usal
8 | Date: 9/9/26
9 | Time: 8:19 p. m.
10 | To change this template use File | Settings | File Templates.
11 | --%>

```

Líneas 4 a 11: Directivas taglib: declaran las librerías **core** (prefijo c) y **fmt** (prefijo fmt) de JSTL, mediante su URI.

```

12 | <%@ page contentType="text/html; charset=UTF-8" language="java" %>
13 | <html>
14 | <head>
15 |     <title>Tabla con JSTL</title>
16 |     <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
17 |         integrity="sha384-sRil4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
17 | </head>

```

Líneas 12 a 17: Directiva page (UTF-8) y apertura de html, head y título.

```

18 | <%
19 |     List<Producto> listaProductos = new ArrayList<>();
20 |     listaProductos.add(new Producto("Teclado Mecanico", 45600.50, true));
21 |     listaProductos.add(new Producto("Mouse Inalambrico", 12800.99, false));
22 |     listaProductos.add(new Producto("Monitor 24' Widescreen", 210000.50, true));
23 |     //pageContext.setAttribute("listaProductos", new ArrayList<>());

```

Líneas 18 a 23: Scriptlet (solo para armar los datos de la demo): crea la lista con tres productos (uno sin stock).

```

24 |     pageContext.setAttribute("productos", listaProductos);
25 | %>

```

Líneas 24 a 25: Publica la lista con el nombre «productos» en el alcance de la página: así la etiqueta `forEach` y `EL` pueden usarla.

```

26 | <body>
27 |     <div class="container">
28 |         <h3 class="text-danger mb-3">Lista de productos</h3>
29 |

```

Líneas 26 a 29: Cuerpo: contenedor y título.

```

30 |         <c:if test="${empty productos}">
31 |             <p class="alert alert-info">No hay elementos en la lista</p>
32 |         </c:if>
33 |

```

Líneas 30 a 33: c:if con el operador **empty**: si la lista está vacía o es null, muestra un aviso.

```

34 |         <c:if test="${not empty productos}">
35 |             <table class="table table-border bg-white shadow-sm">
36 |                 <thead class="table-dark">
37 |                     <tr>
38 |                         <th>Producto</th>
39 |                         <th>Precio</th>
40 |                         <th>Estado</th>
41 |                     </tr>
42 |                 </thead>
43 |                 <tbody>

```

Líneas 34 a 43: Si NO está vacía, dibuja la tabla (encabezado con las tres columnas).

```

44 |                     <c:forEach var="producto" items="${productos}">
45 |                         <tr>
46 |                             <td>${producto.nombre}</td>
47 |                             <td>

```

Líneas 44 a 47: c:forEach: recorre la lista; en cada vuelta la variable `producto` es el elemento actual. **`\${producto.nombre}`** imprime el nombre (llama a `getNombre()`).

```

48 |                             <fmt:formatNumber value="${producto.precio}" type="currency" currencySymbol="$" />
49 |                         </td>
50 |                     </tr>

```

Líneas 48 a 50: fmt:formatNumber: formatea el precio como **moneda** con símbolo «\$».

```

51 |                 <c:choose>
52 |                     <c:when test="${producto.enStock}">
53 |                         <span class="badge bg-success">Disponible</span>
54 |                     </c:when>
55 |                     <c:otherwise>
56 |                         <span class="badge bg-danger">No disponible</span>
57 |                     </c:otherwise>
58 |                 </c:choose>
59 |             </td>
60 | </tr>

```

Líneas 51 a 60: c:choose / c:when / c:otherwise (if-else): si **`${producto.enStock}`** es verdadero muestra un distintivo verde «Disponible»; en otro caso, uno rojo «No disponible».

```

61 |                 </c:forEach>
62 |             </tbody>
63 |         </table>
64 |     </c:if>
65 |
66 | </div>
67 |
68 | </body>
69 | </html>

```

Líneas 61 a 69: Cierra el bucle, la tabla y el c:if; cierra el contenedor, body y html.

Resumen para estudiar

- **Cookie:** dato en el navegador. new Cookie → setMaxAge → response.addCookie; leer con request.getCookies() (puede ser null); borrar con maxAge 0.
- Valores de cookie: codificar con URLEncoder. Nunca guardar contraseñas.
- **JSTL:** reemplaza scriptlets. Directiva taglib + prefijo. Core: if, forEach, choose/when/otherwise, set, out. fmt: formatNumber. EL: \${objeto.propiedad} llama al getter.
- EL solo ve atributos de page, request, session y application (no variables locales de scriptlet).
- En el parcial: el servlet deja los datos en el request y la JSP los muestra solo con JSTL/EL.

Preguntas típicas

¿Cuál es la diferencia entre Session y Cookie?

La Session guarda los datos en el servidor y al cliente solo le envía un id; la cookie guarda el dato en el navegador. La session es más segura; la cookie puede durar días aunque se cierre el navegador.

¿Cómo se borra una cookie?

Se envía una cookie con el mismo nombre y setMaxAge(0).

¿Por qué usar JSTL y no scriptlets?

Porque separa la presentación de la lógica, el código queda más limpio y legible, es más fácil de mantener, y evita mezclar Java con HTML en la vista.